ACTIVIDAD 4 · TECHSTORE

Componentes de Lógica de Negocio · Unidad 4 · Python + FastAPI + pytest

Nombre(s): Carlos Hernan Garza Torres Oscar Denilson García Vélez Edwin Benjamín Ramírez Nava	
Matrícula(s): 1986937 1994273 1956899	
Nombre del curso: Implementación del Núcleo de Negocio	
Nombre del profesor: Miguel Ángel Gómez Marroquin	
Módulo / Actividad: Unidad 4, TechStore	28/10/2026

Bibliografía / fuentes: CLN_U4_Practica_Guiada_Alumnos_TechStore. CLN_U4_Manual_Alumnos_Ilustrado.

BITACORA_E01_E15.md.

1. Objetivo y alcance

Se implementa un sistema de pedidos para una tienda ficticia. Los clientes, productos, pedidos y detalles se cargan desde CSV y se convierten en objetos de dominio. La solución contiene Value Objects, invariantes, Specification Pattern, una máquina de estados, Domain Events, API REST y pruebas automáticas. El almacenamiento es en memoria; no hay base de datos ni control de inventario.

2. Ejecución de la API

Los siguientes pasos son necesarios para la ejecución de la Api y del pytest

1. py –version
2. py -m venv .venv
3. .\.venv\Scripts\Activate.ps1
4. py -m pip install -r requirements.txt
5. .\.venv\Scripts\python.exe -m uvicorn app.api:app --host 127.0.0.1 --port 8000
6. py -m pytest -q

2. Reglas de negocio implementadas

- Money usa Decimal y rechaza importes negativos o no finitos; no permite sumar monedas diferentes.
- Máximo 20 unidades por pedido: se suman cantidades, no renglones.
- Descuento 10%: cliente VIP, total > $500 MXN, al menos 4 unidades y sin promoción.
- Descuento 5%: total > $5,000 MXN y sin promoción. La regla del 10% tiene prioridad si ambas se cumplen.
- Todos los pedidos cargan en DRAFT. Solo cliente activo con pedido no vacío puede confirmar.
- Enviar requiere CONFIRMED y guía; cancelar requiere CONFIRMED y motivo; entregar/devolver requiere SHIPPED.
- Una operación inválida no modifica estado ni publica un evento.

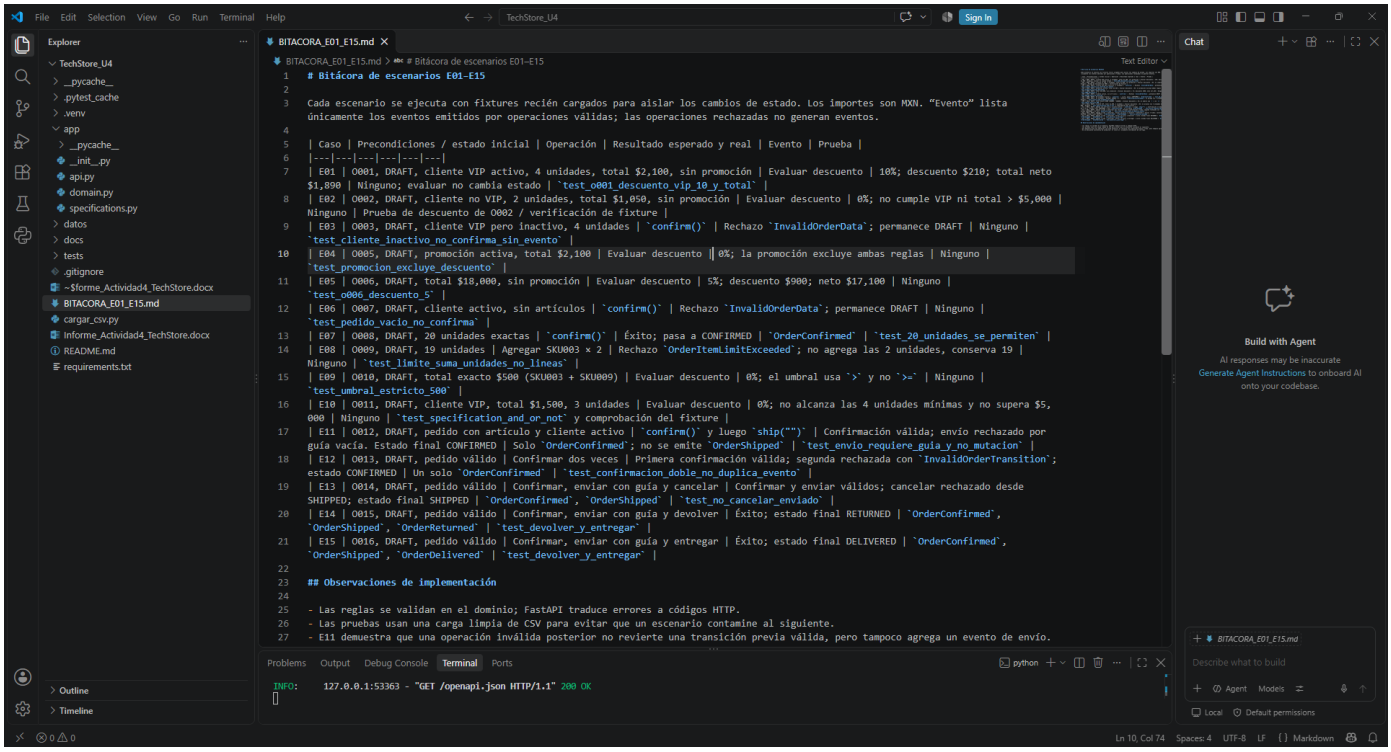
3. Arquitectura del proyecto

Módulo	Responsabilidad
app/domain.py	Customer, Money, OrderItem, Order, invariantes, estados y eventos.
app/specifications.py	Specifications de monto, cliente VIP, unidades y promoción; composición AND/OR/NOT.
app/api.py	API REST FastAPI; transforma errores de dominio a HTTP 404, 409 y 422.
cargar_csv.py + datos/	Carga reproducible de los 16 pedidos del paquete.
tests/	Pruebas de reglas, transiciones, eventos y API.

4. Bitácora de escenarios E01–E15

Cada caso se ejecuta con una carga limpia de fixtures. La prueba asociada comprueba resultado, estado y eventos cuando aplica.

Caso	Operación	Resultado	Evento(s)	Prueba principal
E01	O001 · descuento	10%; $210 descuento, neto $1,890	Sin evento; evaluación no muta	test_o001_descuento_vip_10_y_total
E02	O002 · descuento	0%; no es VIP y no supera $5,000	Sin evento	test_e02_e10_sin_descuento_por_condiciones
E03	O003 · confirmar	Rechazo: cliente inactivo; sigue DRAFT	Ninguno	test_cliente_inactivo_no_confirma_sin_evento
E04	O005 · descuento	0%; promoción activa bloquea descuento	Sin evento	test_promocion_excluye_descuento
E05	O006 · descuento	5%; $900 descuento, neto $17,100	Sin evento	test_o006_descuento_5
E06	O007 · confirmar	Rechazo: pedido vacío; sigue DRAFT	Ninguno	test_pedido_vacio_no_confirma
E07	O008 · confirmar	Éxito; 20 unidades permitidas, CONFIRMED	OrderConfirmed	test_20_unidades_se_permiten
E08	O009 · agregar 2 unidades	Rechazo: 19 + 2 > 20; conserva 19	Ninguno	test_limite_suma_unidades_no_lineas
E09	O010 · descuento	0%; total exacto $500 no supera umbral	Sin evento	test_umbral_estricto_500
E10	O011 · descuento	0%; solo 3 unidades	Sin evento	test_e02_e10_sin_descuento_por_condiciones
E11	O012 · confirmar/enviar sin guía	Confirmar sí; envío no; queda CONFIRMED	OrderConfirmed únicamente	test_e11_confirmacion_valida_envio_invalido_conserva_evento
E12	O013 · confirmar dos veces	Primera válida; segunda rechazada; CONFIRMED	Un OrderConfirmed	test_confirmacion_doble_no_duplica_evento
E13	O014 · confirmar/enviar/cancelar	Cancelar rechazado desde SHIPPED	OrderConfirmed, OrderShipped	test_e13_e14_e15_transiciones_y_eventos
E14	O015 · devolver	Éxito; termina RETURNED	OrderConfirmed, OrderShipped, OrderReturned	test_devolver_y_entregar
E15	O016 · entregar	Éxito; termina DELIVERED	OrderConfirmed, OrderShipped, OrderDelivered	test_devolver_y_entregar



5. Máquina de estados

DRAFT → CONFIRMED → SHIPPED → DELIVERED. Desde CONFIRMED también puede transitar a CANCELLED; desde SHIPPED puede transitar a RETURNED. No se permiten saltos ni transiciones inversas. El diagrama Mermaid y la definición de los cinco eventos se encuentran en docs/estados.md.

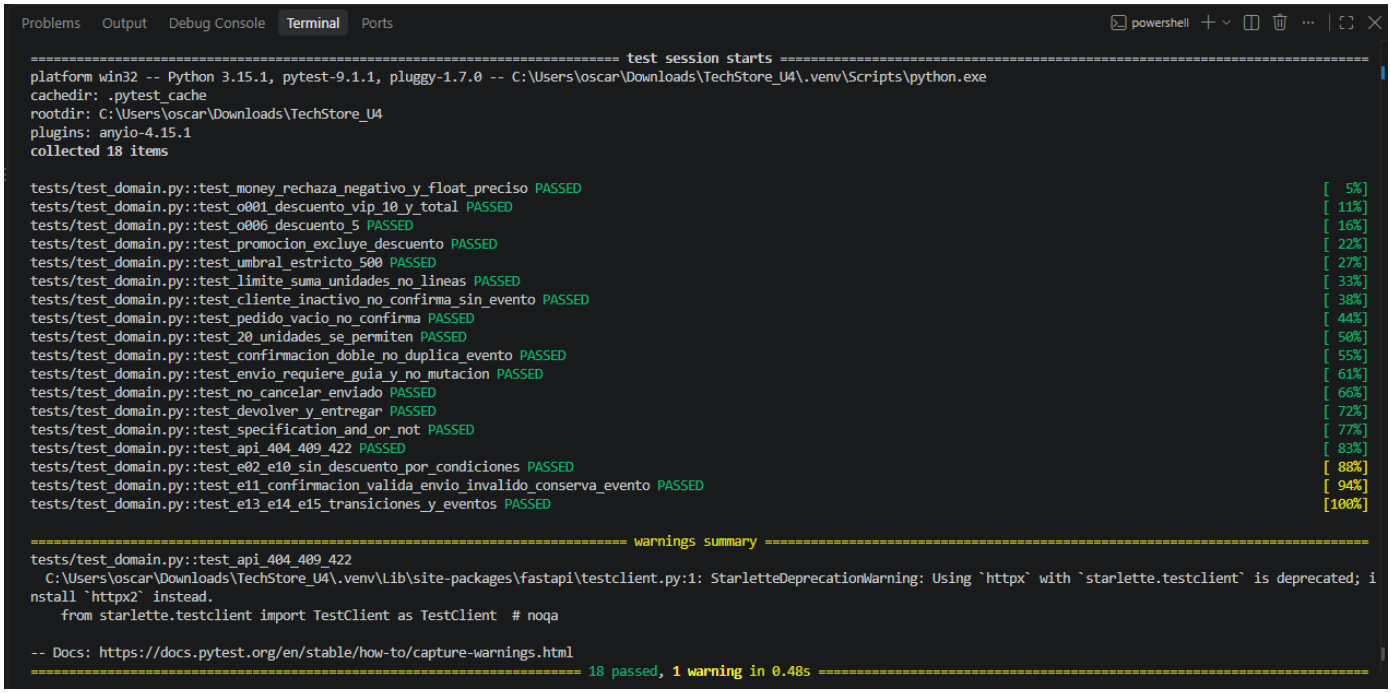
6. API REST

- GET /health
- GET /orders y GET /orders/{id}
- POST /orders/{id}/confirm
- POST /orders/{id}/ship con tracking_number
- POST /orders/{id}/cancel con reason
- POST /orders/{id}/deliver
- POST /orders/{id}/return

La documentación interactiva de OpenAPI se publica en /docs. El servidor carga los fixtures al iniciar y los cambios se pierden al reiniciar.

7. Ejecución y resultado de pruebas

Instalación: python -m pip install -r requirements.txt. Carga: python cargar_csv.py. Pruebas: python -m pytest -q. Resultado verificado durante la preparación de esta entrega: 18 passed. La carga inicial imprime 16 pedidos en estado DRAFT; O001 muestra 4 unidades y $2,100.00 MXN.



8. Reflexión técnica

La primera decisión importante fue representar dinero con Decimal para evitar errores de precisión binaria. El límite de pedido debe calcular unidades totales y validar antes de modificar la lista; contar líneas no satisface la regla. La máquina de estados centraliza las transiciones y los Domain Events se agregan solo después de una operación válida, por lo que as pruebas pueden detectar mutaciones parciales.

9. Enlace de GitHub

aquí se encuentra el código completo de la API y la bitácora utilizada

https://github.com/OscarDenilson/Componentes-de-L-gica-de-Negocio/upload/main/UNIDAD%204

10. Conclusión general

La realización de la Actividad 4 de TechStore permitió comprender la importancia de desarrollar y probar una aplicación mediante la implementación de reglas de negocio, el diseño de una API REST y la ejecución de pruebas automatizadas. A través de los escenarios E01–E15, se verificó el comportamiento de los pedidos, los descuentos, las transiciones de estado y el manejo de operaciones inválidas. Asimismo, el uso de Python, FastAPI y pytest permitió relacionar los conocimientos teóricos con una aplicación funcional, destacando la importancia de validar los datos, controlar los errores y mantener la integridad de la información. En conclusión, esta actividad fortaleció las habilidades de programación, análisis, pruebas de software y trabajo en equipo, fundamentales para desarrollar sistemas confiables y fáciles de mantener.

11. Conclusiones personales

1. Carlos Hernan Garza Torres

La realización de esta actividad me permitió comprender mejor cómo se aplican las reglas de negocio dentro de un sistema de software y por qué es importante comprobar que cada operación funcione correctamente. Mediante los escenarios de prueba de TechStore, pude identificar la importancia de validar los pedidos, calcular los descuentos de manera adecuada y evitar operaciones que no cumplen con las condiciones establecidas. También comprendí que las pruebas automatizadas ayudan a detectar errores y a mejorar la confiabilidad de una aplicación. Considero que los conocimientos adquiridos serán útiles para desarrollar proyectos más completos y organizados en el futuro.

2. Oscar Denilson García Vélez

Esta actividad me ayudó a reforzar mis conocimientos sobre el desarrollo de aplicaciones con Python y FastAPI, así como a comprender cómo se relacionan las reglas de negocio con las operaciones que puede realizar un usuario. Trabajar con TechStore me permitió reconocer la importancia de probar diferentes situaciones, incluyendo los casos en los que una operación debe ser rechazada. También aprendí que documentar los resultados mediante una bitácora facilita la revisión del sistema y permite comprobar que se cumplen los requisitos establecidos. En general, considero que esta práctica fue una experiencia valiosa para mejorar mis habilidades de programación y resolución de problemas.

3. Edwin Benjamín Ramírez Nava

El desarrollo de la Actividad 4 me permitió comprender la importancia de realizar pruebas sistemáticas para verificar el funcionamiento de una aplicación. A través de los escenarios de TechStore, pude apreciar que no basta con programar las funciones principales, sino que también es necesario contemplar errores, restricciones y cambios de estado para evitar comportamientos incorrectos. Además, comprendí la utilidad de herramientas como pytest para automatizar las verificaciones y facilitar el mantenimiento del código. Esta experiencia me permitió fortalecer mis conocimientos sobre la calidad del software y reconocer la importancia de trabajar de manera organizada y colaborativa en un proyecto de programación.